TECHAINER

Review kỹ thuật hệ thống Horus · Giám sát huấn luyện AI

Stream giật / trễ, độ phủ yêu cầu, tối ưu, chất lượng AI, vận hành, UI/UX responsive và lộ trình cải thiện

Ngày	**02/11/2026**
Repo	`horus/military-attendance-check`
Commit review	**local `6fc2929` · đang deploy `ac6f28b` (chậm 3 commit, chỉ khác Docker và script probe; mã app giống nhau)**
Môi trường đo	`techainer-rtx3090-1` **· https://military-attendance-dev.techainer.com**
Yêu cầu đối chiếu	**FunctionList_AI Giám Sát & Chat Bot_v0.3.xlsx**
Đối tượng đọc	**Team dev nội bộ**
Phương pháp	**Đọc code, đo trực tiếp trên server (chỉ đọc), đo stream qua Cloudflare, chụp UI ở 3 kích thước màn hình**

NỘI BỘ · Tài liệu có nêu lỗ hổng chưa vá — không gửi ra ngoài. Mọi mật khẩu/token đã được che.

Mục lục

1. Tóm tắt điều hành
2. Bảo mật khẩn cấp
3. Stream: giật, đứng hình và trễ — số đo & nguyên nhân
4. Kỹ thuật stream: có cần WebRTC?
5. Tối ưu: fix nhanh và tái kiến trúc
6. Đánh giá AI
7. Ma trận yêu cầu (FunctionList v0.3)
8. Vận hành, dữ liệu, triển khai
9. UI/UX & responsive
10. Lộ trình & tiêu chí nghiệm thu
11. Phụ lục: số liệu gốc & cách đo

1. Tóm tắt điều hành

0.15–0.25
fps thực tới trình duyệt (camera phát 25 fps, cấu hình 5 fps)

3.0 s
tuổi trung bình của hình khi tới trình duyệt (p95 5.4 s, max 6.0 s)

~13 s
độ cũ tệ nhất người xem có thể thấy (tuổi frame + thời gian giữ hình)

19/28
dòng AI Giám sát xong về chức năng; 9 một phần. Phân quyền chưa đạt. Chat Bot 0/63

Kết luận chính. Stream giật **không do Cloudflare**: đo trên localhost của server ra cùng con số, tunnel chỉ cộng ~0.16 s. Gốc rễ nằm ở kiến trúc: **hình chỉ được phát ra sau khi AI xử lý xong frame đó**, trong một vòng lặp tuần tự / camera. AI lại đang chạy chủ yếu trên CPU (onnxruntime bản CPU, giải mã HEVC bằng CPU) trên một máy 8 core đang chạy chung job train của người khác (load ~22). Vì thế: fps thấp (giật), thỉnh thoảng đứng hẳn vài giây (ffmpeg ghi clip và ghi file chặn vòng lặp), và trễ 3–13 s (người đi qua rồi mới hiện).

HẠNG MỤC	ĐÁNH GIÁ	NHẬN XÉT NGẮN
Bảo mật	Nghiêm trọng	API public không auth; `GET /api/v1/cameras` trả mật khẩu RTSP ra Internet; mọi API ghi/xoá mở; token frp nằm trong git.
Hiệu năng stream	Nghiêm trọng	0.15–0.25 fps, trễ 3 s trung bình, đứng hình 5–7 s. Video bị trói vào tốc độ AI.
Vận hành	Nghiêm trọng	Chạy tay trong tmux, không tự khởi động lại; `data/events` 166 GB, đĩa 91%, không có retention.
Chất lượng AI	Cần cải thiện	Pipeline hợp lý (YOLO11 + ByteTrack + SCRFD/ArcFace) nhưng: tracker cấu hình 30 fps khi chạy <1 fps, ngưỡng mặt 0.30 quá thấp, không reconnect, xâm nhập không có dwell/hysteresis, không chống giả mạo ảnh.
Độ phủ yêu cầu	Khá nếu Chat Bot ngoài phạm vi Thấp (19/91) nếu trong phạm vi	AI Giám sát: 19/28 xong về chức năng, 9 một phần (giám sát tự động theo lịch, khu vực là nhóm camera, khu vực trường bắn, màn tổng hợp quân số, tiến độ hoàn thành). Không dòng nào đạt phân quyền ở backend. Chat Bot: 0/63, chưa có code; cần xác nhận phạm vi.
UI/UX	Cần cải thiện	Desktop ổn; mobile/tablet vỡ (nội dung rộng tới 1359 px ở màn 375 px). Gần như không có a11y. Thiếu trạng thái "hình cũ/mất tín hiệu".
Chất lượng code & test	Khá	Có 600+ check (API, AI logic, UI jsdom). Chưa test vòng video end-to-end; test ghi đè `data/*.json` thật; JSON storage không khoá.

5 việc làm ngay (tuần này)

1. **Khoá truy cập public**: bật Cloudflare Access cho domain, tắt đường frpc, ẩn `source_uri` khỏi API, **đổi mật khẩu 2 camera** và token frp.
2. **Dọn & đặt retention** cho `data/events` (166 GB, đĩa 91% — sắp đầy sẽ làm chết cả app lẫn job khác trên máy).
3. **Fix nhanh stream**: phát hình thô ngay sau capture (không chờ AI), đẩy ffmpeg và ghi file ra nền, bỏ các model thừa, cho AI đọc luồng phụ thay vì 1080p. Máy thật của khách **chỉ có CPU**, nên dùng OpenVINO và cấu hình tự thích ứng phần cứng (mục 5.3–5.4). `onnxruntime-gpu` chỉ dành cho máy dev.

4. **Chạy app bằng Docker/systemd** có restart, giới hạn CPU, tách khỏi job train (hoặc dời job train).
5. **Bắt đầu PoC go2rtc** (WebRTC + fallback MSE qua WebSocket) và overlay vẽ phía trình duyệt — đây là lời giải dứt điểm cho giật và trễ.

2. Bảo mật khẩn cấp

Đang lộ ra Internet ngay lúc viết báo cáo

Đã kiểm chứng: gọi `GET https://military-attendance-dev.techainer.com/api/v1/cameras` không cần đăng nhập trả về `source_uri` dạng `rtsp://user:<mật khẩu>@192.168.1.x/...` cho cả 2 camera. Camera nằm trong LAN nên chưa bị truy cập trực tiếp từ ngoài, nhưng mật khẩu đã lộ và cần đổi.

#	VẤN ĐỀ	BẰNG CHỨNG	MỨC	CÁCH XỬ LÝ
S1	Lộ mật khẩu RTSP qua API	`app/api.py:1343` `_camera_out` trả nguyên `**camera`; hàm `mask_credentials` có sẵn ở `video_processor.py:140` nhưng không dùng ở đây	P0	Che bằng `mask_credentials` khi trả ra; PATCH giữ mật khẩu cũ nếu client gửi bản đã che. Đổi mật khẩu camera.
S2	Không có auth phía server	`app/auth.py:1-10` ghi rõ "không phải ranh giới bảo mật"; role chỉ lưu ở `localStorage` (`app.js:3317`) — sửa tay là thành quản trị	P0	Ngắn hạn: Cloudflare Access (SSO email @techainer). Dài hạn: token (JWT/cookie phiên) + dependency kiểm role trên từng router; SSE/MJPEG nhận token qua cookie.
S3	Mọi API ghi/xoá mở công khai	start/stop camera, đăng ký/xoá khuôn mặt, zone, lịch, ack sự kiện; ảnh khuôn mặt, ảnh/clip sự kiện tải tự do	P0	Như S2; static `/data/*` phải qua kiểm quyền.
S4	Cửa public thứ 2 qua frp	Container `military-attendance-check-frpc-1` publish cổng 8199 ra `tunnel.techainer.com`	P0	Tắt nếu không dùng; nếu dùng thì cùng lớp bảo vệ.
S5	Token frp commit trong git	`frpc.ini:4`	P1	Rotate token, chuyển sang biến môi trường, xoá khỏi lịch sử nếu repo chia sẻ ra ngoài.
S6	Mật khẩu demo in trên màn đăng nhập	`static/index.html:52` khối "Tài khoản demo"; README	P1	Bỏ khỏi UI; ép đổi mật khẩu lần đầu.
S7	Mật khẩu camera dạng chữ trong tài liệu trên server	`docs/camera_ai_stream_guide.md` (file chưa track trên máy deploy)	P1	Che mật khẩu trong tài liệu; giữ creds trong secret store/.env.
S8	HSTS tắt, không giới hạn upload/rate	header `hsts max-age=0`; `/api/upload_chunk`	P2	Bật HSTS ở Cloudflare; giới hạn kích thước & tần suất.

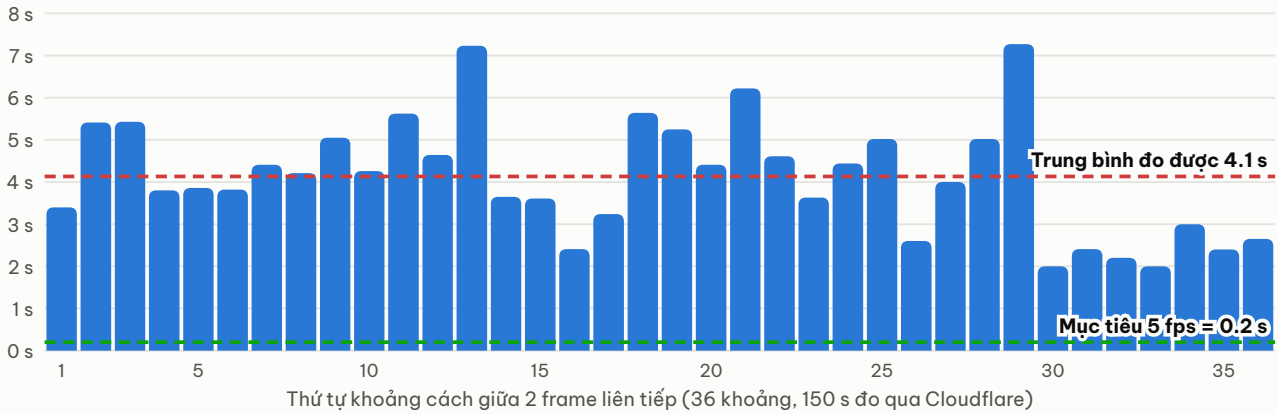
3. Stream: giật, đứng hình và trễ

"Giật" mà người dùng báo thực chất là **3 triệu chứng khác nhau**, mỗi cái có nguyên nhân riêng:

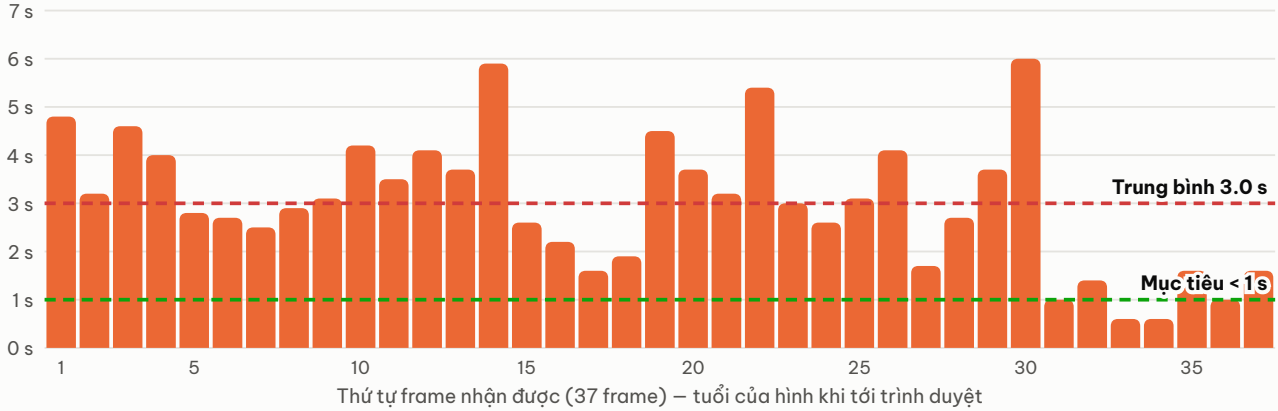
TRIỆU CHỨNG	SỐ ĐO	NGUYÊN NHÂN CHÍNH
(a) FPS thấp — hình nhảy cóc	0.15–0.25 fps; khoảng cách giữa 2 frame TB 4.1 s, min 2.0 s, max 7.3 s	Vòng xử lý tuần tự: YOLO@1280 + SCRFD@960 + quét lại ≤4 crop + vẽ + 4 lần encode + sleep, chạy chủ yếu trên CPU đang quá tải.
(b) Lâu lâu đứng hẳn	khoảng trống 5–7 s; đứng tới khi ffmpeg xong mỗi lần có sự kiện	`await self._archive_clip()` (ffmpeg libx264 60 frame) ngay trong vòng (`video_processor.py:694, 807`); `events.emit` ghi ảnh + JSON trong event loop; `probe_stream` join 8 s và load YOLO khi bấm start (`api.py:1444, 1452`) làm treo **mọi** stream; mất RTSP thì đứng ở frame cũ vĩnh viễn.
(c) Trễ — "đi qua rồi mới hiện"	tuổi hình khi tới: TB 3.0 s, p95 5.4 s; cộng thời gian giữ hình ⇒ cũ tới ~13 s	Hình được publish **sau** khi AI chạy xong; MJPEG poll 200 ms; sau đó hình đứng yên tới frame kế.

3.1 Số đo thực tế (02/11/2026, 18:02–18:04, cam_01 "Sân tập trung")

Cách đo: kéo `stream.mjpg?overlay=1` qua Cloudflare 150 s, ghi giờ nhận từng frame; đọc đồng hồ OSD camera in trên hình. Đồng hồ camera đang **chậm 6 phút 42 giây** — đã hiệu chỉnh bằng cách lấy keyframe trực tiếp từ RTSP trên server (NTP đồng bộ cả 2 đầu). Sai số ±1 s do OSD chỉ có giây.



Hình 3.1 — Khoảng cách giữa 2 frame liên tiếp. Mục tiêu 0.2 s (5 fps); camera thật phát 0.04 s (25 fps) ⇒ trình duyệt chỉ thấy ~1/100 số frame.

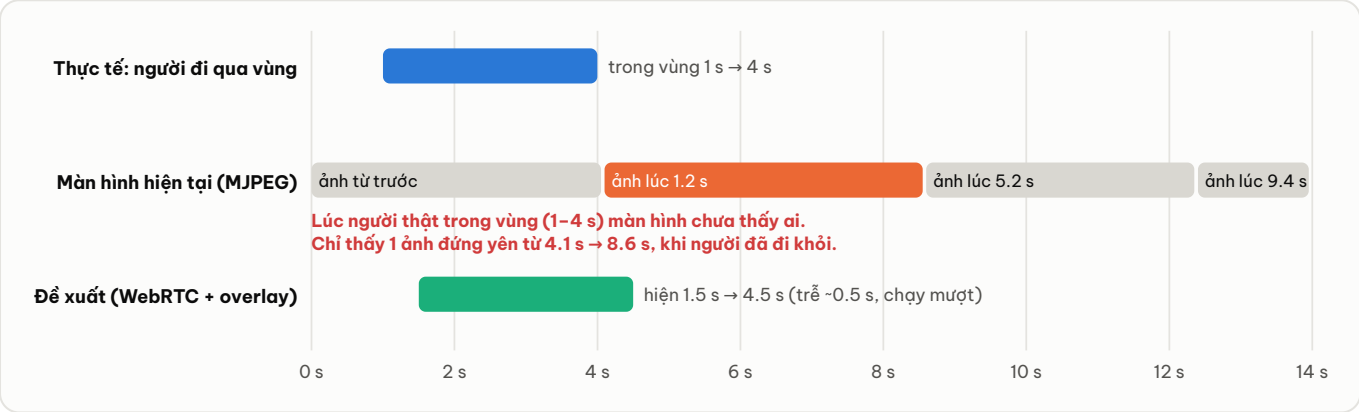


Hình 3.2 — Tuổi của hình khi tới trình duyệt (glass-to-glass, chưa tính thời gian hình đứng chờ frame kế).

CHẶNG	TB	TRUNG VỊ	P95	MAX
Camera chụp → xong AI (timestamp app vẽ ở `video_processor.py:551`)	1.7 s	2.0 s	3.0 s	5.0 s
Xong AI → trình duyệt nhận (vẽ, encode, poll, Cloudflare)	1.3 s	1.1 s	2.7 s	5.0 s
Glass-to-glass	**3.0 s**	3.0 s	5.4 s	6.0 s
Khoảng cách giữa 2 frame	4.1 s	4.1 s	6.2 s	7.3 s

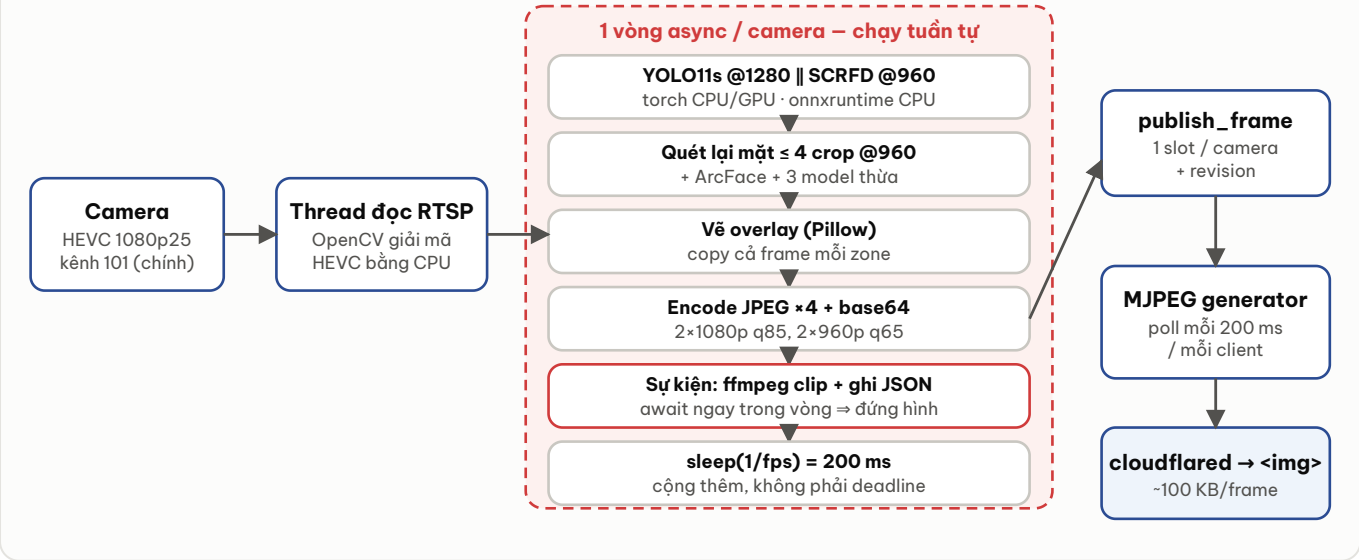
Cloudflare không phải thủ phạm
Đo song song trên `localhost:8199` của server: 0.15 fps, khoảng cách 6.5 s ± 2.3 s — giống hệt qua Cloudflare. TTFB qua tunnel chỉ +0.16 s. Băng thông ~120 kbps/stream (frame ~100 KB ở 960 px, q65).

3.2 Vì sao "người đi qua rồi mới hiện"

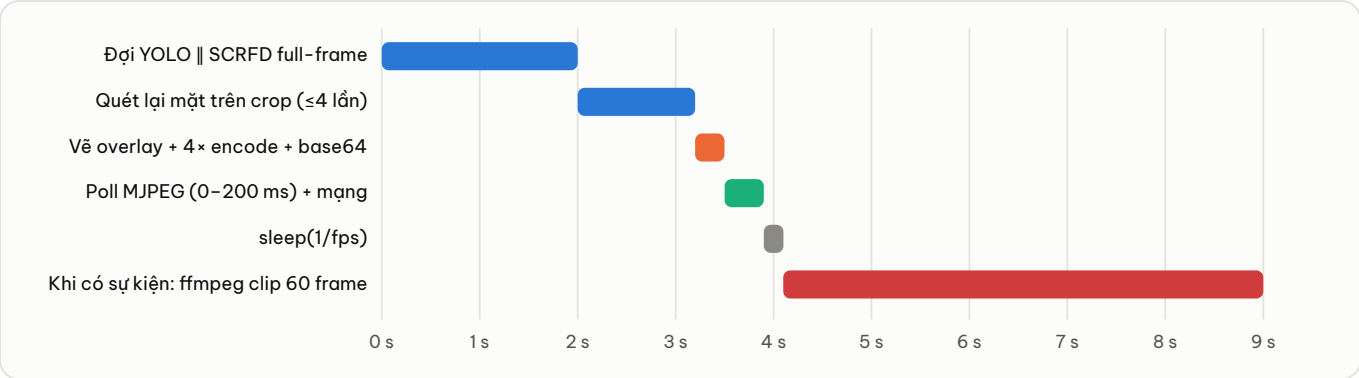


Hình 3.3 — Minh hoạ dựa trên số đo (frame cách nhau ~4 s, tuổi ~3 s). Với MJPEG hiện tại, một người đi qua vùng trong 3 s có thể chỉ xuất hiện trên 1 tấm hình đứng yên, và tấm đó hiện ra sau khi người đã rời đi.

3.3 Kiến trúc hiện tại: video bị trói vào AI



Hình 3.4 — Luồng hiện tại. Mọi bước trong khung đỏ chạy nối tiếp trong 1 coroutine / camera trên 1 event loop chung với API, SSE và MJPEG.



Hình 3.5 — Một vòng xử lý (ước tính, chia từ chu kỳ đo được ~4.1 s; chưa có instrument trong code — xem đề xuất đo ở mục 5). Thanh đỏ chỉ xuất hiện khi có sự kiện, là nguyên nhân các lần đứng hình dài.

3.4 Nguyên nhân xếp theo ảnh hưởng

#	NGUYÊN NHÂN	VỊ TRÍ	ẢNH HƯỞNG
1	Publish hình nằm cuối vòng AI tuần tự; `sleep(1/fps)` cộng thêm thay vì deadline	`video_processor.py:480-570, 753-760, 823`	FPS + trễ
2	InsightFace chạy CPU: venv cài `onnxruntime` (CPU), log xác nhận `Applied providers: ['CPUExecutionProvider']`; `pyproject.toml` pin `onnxruntime`	`face_engine.py:31-41`, `pyproject.toml:17`	FPS
3	Mô hình nặng không cần thiết: YOLO `imgsz=1280`; SCRFD `det_size=960` trên cả frame **và** ≤4 crop phóng lên 960; buffalo_l chạy cả 3 model thừa (landmark 2D/3D, giới tính/tuổi) do thiếu `allowed_modules`	`detector.py:38`, `face_engine.py:40`, `video_processor.py:566`	FPS
4	Giải mã HEVC 1080p25 bằng CPU (OpenCV/FFmpeg), dù chỉ dùng ~0.2 frame/s; NVDEC đang 0%	`video_processor.py:147-151, 222-234`	CPU
5	ffmpeg ghi clip được `await` ngay trong vòng; `events.emit` ghi ảnh + JSON đồng bộ	`video_processor.py:694, 807`; `events.py:141, 185-200`	Đứng hình
6	Hàm chặn trong handler async: `probe_stream` (join 8 s), khởi tạo YOLO	`api.py:1444, 1452`; `video_processor.py:175`	Đứng toàn server
7	Không giới hạn thread: torch + ORT mỗi bên dùng tất cả core, nhân theo số camera; máy 8 core load ~22 (chung job train PaddleOCR)	không có `set_num_threads` / `SessionOptions`	Jitter
8	4 lần encode JPEG + base64 full-res mỗi vòng, broadcast WS mà frontend không còn dùng	`video_processor.py:753-780`, `api.py:135-150`	CPU
9	MJPEG generator poll 200 ms thay vì chờ sự kiện frame mới	`api.py:1259-1269`	+0–200 ms jitter
10	Không reconnect RTSP; khi đọc lỗi `self.frame` giữ nguyên ⇒ đứng ở frame cũ mãi, cảnh báo "mất tín hiệu" không bao giờ bật	`video_processor.py:222-234, 454-461`	Đứng hình
11	ByteTrack cấu hình mặc định 30 fps khi chạy 0.2 fps ⇒ đổi ID, hộp nhảy lung tung (cảm giác giật)	`detector.py:85-93`	Cảm nhận
12	MJPEG qua HTTP/2 + cloudflared: không có backpressure, dồn buffer rồi xả cục	kiến trúc	Burst

4. Kỹ thuật stream: có cần WebRTC?

Có — cho phần xem live. Nhưng WebRTC chỉ giải quyết được phần *truyền video*. Nếu vẫn bắt video chờ AI như hiện nay thì đổi sang WebRTC cũng không hết giật. Điều kiện tiên quyết là **tách video khỏi AI**: video đi thẳng từ camera tới trình duyệt, kết quả AI đi đường riêng dưới dạng dữ liệu (toạ độ hộp + timestamp) và được vẽ đè ở trình duyệt.

4.1 Hiện trạng nguồn camera (ffprobe trên server)

CAMERA	LUỒNG	CODEC	ĐỘ PHÂN GIẢI	FPS	GHI CHÚ
cam_01 Hikvision (192.168.1.5)	`Channels/101` (app đang dùng)	HEVC Main	1920×1080	25	giải mã CPU
	`Channels/102`	HEVC Main	640×360	25	đủ cho detect người ở gần/vừa
i-PRO WV-S66600-Z3L (192.168.1.113)	`stream_1..4`	HEVC	6MP / 1080p / 720p / 360p	15	có AI trên camera + ONVIF Profile M metadata; hiện **không tới được** từ server (No route to host)

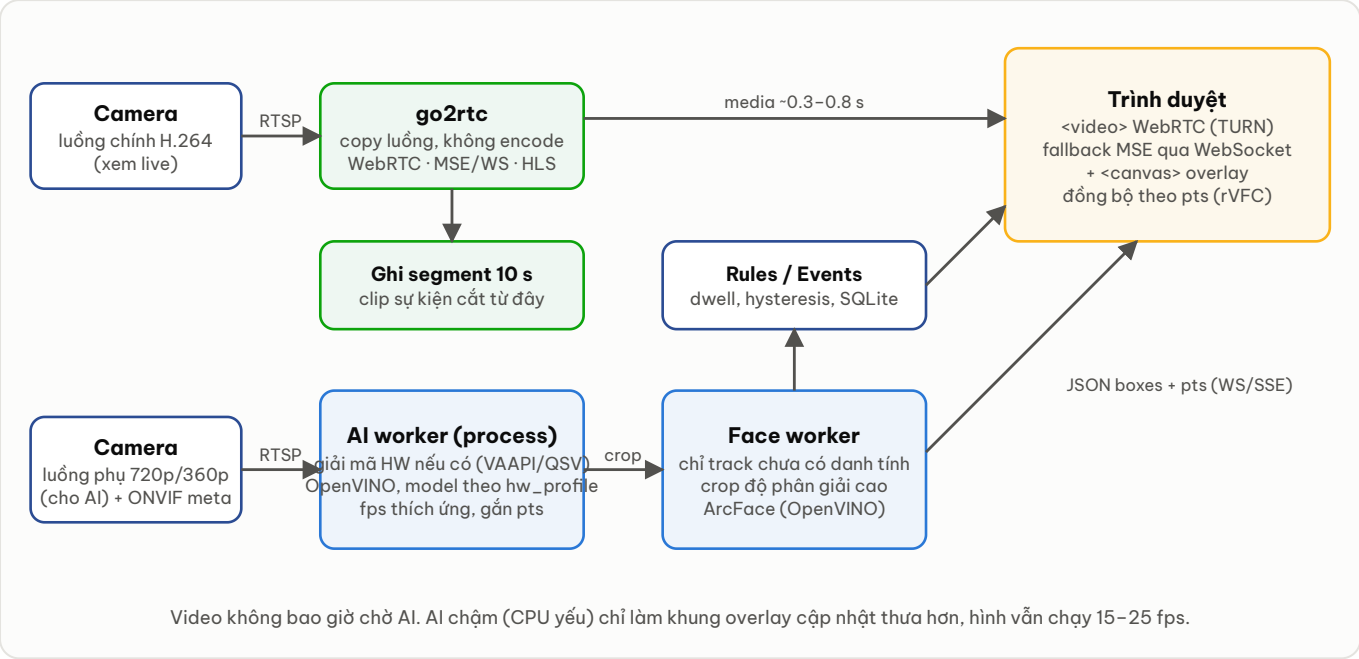
HEVC là rào cản cho trình duyệt
WebRTC/MSE với H.265 chỉ chạy trên Safari và Chrome/Edge bản mới có giải mã phần cứng; Firefox và nhiều máy Windows không có. Khuyến nghị **chuyển luồng dùng để xem sang H.264** trong cấu hình camera (cả Hikvision lẫn i-PRO cho chọn codec theo từng luồng). Nếu không đổi được, transcode bằng NVENC trên RTX 3090 (go2rtc gọi ffmpeg `h264_nvenc`, ~1–2% GPU / luồng 1080p).

4.2 So sánh phương thức truyền

PHƯƠNG THỨC	ĐỘ TRỄ	BĂNG THÔNG 1080P	QUA CLOUDFLARED	HEVC	ĐỘ PHỨC TẠP	ĐÁNH GIÁ
MJPEG (hiện tại)	= tốc độ server	rất cao (~4 Mbps @5 fps 960p)	được, nhưng burst	—	thấp	giữ làm fallback cuối
HLS / LL-HLS	2–6 s / ~1–2 s	thấp (H.264 copy)	tốt (HTTP thường, cache được)	Safari	thấp	xem lại / ghi hình
MSE qua WebSocket (fMP4, go2rtc)	0.5–1 s	thấp	**tốt** (WebSocket qua tunnel)	Chrome/Safari có HW	thấp (go2rtc có sẵn player)	fallback chính
WebRTC (WHEP, go2rtc)	**0.2–0.5 s**	thấp	**không trực tiếp** — media đi UDP; cần TURN (Cloudflare Realtime TURN hoặc coturn trên VPS public) hoặc mở UDP	Safari, Chrome mới	trung bình (TURN, ICE)	chính cho live

Khuyến nghị: dùng **go2rtc** (một binary, có Docker image): nhận RTSP từ camera, **không encode lại**, phát đồng thời WebRTC + MSE/WebSocket + HLS + MJPEG; player `<video-stream>` của go2rtc tự thử WebRTC rồi lùi về MSE. Khi đi qua cloudflared mà chưa có TURN, người dùng vẫn được MSE ~0.5–1 s — đã tốt hơn hiện tại ~10 lần. Có thể dùng mediamtx thay thế; go2rtc gọn hơn cho trường hợp này.

4.3 Kiến trúc đề xuất



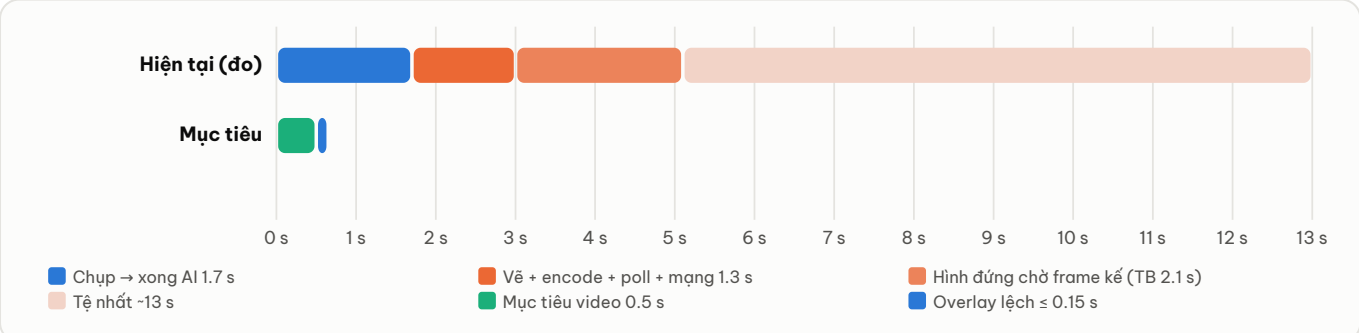
Hình 4.1 — Tách 2 đường: **video** (camera → go2rtc → trình duyệt) và **AI** (luồng phụ → worker GPU → JSON có timestamp → canvas). Clip sự kiện cắt từ bản ghi segment, không giữ 60 frame base64 trong RAM.

Đồng bộ overlay với video

- AI worker gắn `pts` (RTP timestamp / thời điểm capture) vào mỗi kết quả: `{camera_id, pts, tracks:[{id, box, label, identity, zone_state}]}`, đẩy qua WebSocket (hoặc SSE hiện có).
- Trình duyệt dùng `video.requestVideoFrameCallback()` (có `mediaTime`, với WebRTC có cả `rtpTimestamp`) để chọn kết quả AI khớp frame đang hiển thị; nếu kết quả tới sau video, nội suy/dự đoán vị trí hộp theo vận tốc track (Kalman đơn giản) — mắt người không thấy độ lệch <150 ms.
- Ảnh bằng chứng/snapshot sự kiện vẫn render overlay phía server (chỉ khi có sự kiện), không cần cho mỗi frame.

Tận dụng AI trên camera i-PRO (PoC)

Camera i-PRO đã chạy sẵn phát hiện người + tracking ID và AI-VMD (xâm nhập, vượt rào ảo, lảng vãng), xuất **ONVIF Profile M metadata** trong cùng phiên RTSP (`?analytics=1`), cùng đồng hồ RTP với video. Nếu PoC đạt độ chính xác chấp nhận được (dev đã có `scripts/probe_ai_camera.py`), server chỉ còn làm **nhận diện khuôn mặt** cho track mới ⇒ giảm phần lớn tải GPU/CPU và overlay gần như không trễ. Cần giữ đường YOLO cho camera không có AI (Hikvision hiện tại).



Hình 4.2 — Ngân sách độ trễ: hiện tại (đo) so với mục tiêu. Mục tiêu: video p95 < 1 s, overlay lệch video ≤ 200 ms.

5. Tối ưu: fix nhanh và tái kiến trúc

Hiện code đã có một số điểm tốt: thread đọc RTSP chỉ giữ frame mới nhất (tự bỏ frame cũ), MJPEG encode 1 lần và chia sẻ cho mọi client, YOLO và nhận diện mặt chạy song song bằng `to_thread`. Chưa tối ưu ở: dùng sai phần cứng, chọn kích thước model, và trói video vào AI.

5.1 Fix nhanh (P0, 2–4 ngày công, chưa đổi kiến trúc)

#	THAY ĐỔI	CHỖ SỬA	KỲ VỌNG
Q1	**Tách 2 nhịp**: một task 10–15 fps lấy frame mới nhất từ reader, vẽ *kết quả AI gần nhất*, encode 1 lần ở 960 px rồi publish; vòng AI chạy riêng và chỉ cập nhật kết quả	`video_processor.py` (tách phần 493–760 khỏi 480–570)	stream 10–15 fps ngay cả khi AI 1 fps; trễ video ~0.3–0.6 s
Q2	Runtime theo phần cứng: máy thật CPU-only dùng **OpenVINO** (mục 5.3); máy dev có GPU thì dùng `onnxruntime-gpu`. Khi khởi động, log rõ đang chạy trên thiết bị nào	`pyproject.toml`, `Dockerfile`, `face_engine.py`	CPU: R50 nhanh khoảng 3× so với ORT CPU (đo). GPU: nhanh khoảng 10–20×
Q3	`FaceAnalysis(allowed_modules=['detection','recognition'])`; `FACE_DET_SIZE=640` cho cả frame, 320–480 cho crop; rescan tối đa 1–2 và chỉ cho track chưa có danh tính	`face_engine.py:66`, `video_processor.py:120, 336-357`	bớt 3 model/mặt, bớt ~50–70% thời gian face
Q4	YOLO11**s** chạy 640 trên **ROI** thay vì 1280 trên cả khung hình. **Không hạ xuống YOLO11n** với camera góc rộng: đo được chỉ còn bắt 18% số người (hình 5.2). 1 model dùng chung cho mọi camera	`detector.py:38, 85`	Nhanh khoảng 6× (642 → 96 ms ở mức AVX2)
Q5	AI đọc luồng phụ (102 / stream_3); giải mã phần cứng nếu máy có (VAAPI/QSV, máy dev thì NVDEC). Chỉ kéo luồng chính khi cần crop mặt độ phân giải cao, ví dụ trong cửa sổ điểm danh	cấu hình camera, `open_stream_capture`	H.265 1080p 0.17 → 360p 0.018 nhân (đo, ban đêm)
Q6	`_archive_clip`, `events.emit`, ghi JSON, `monitor` chạy nền (`asyncio.create_task(asyncio.to_thread(...))`); `probe_stream` và khởi tạo `CameraRuntime` qua `to_thread`	`video_processor.py:694, 807`; `api.py:1444, 1452`	hết đứng hình theo sự kiện
Q7	Giới hạn thread: `torch.set_num_threads(2)`, ORT `intra_op_num_threads=2`, `OMP_NUM_THREADS`; giới hạn CPU cho container	khởi tạo model	giảm jitter, không giẫm nhau
Q8	Bỏ encode full-res + base64 mỗi frame và broadcast `frame_update` qua WS; clip buffer lưu bytes JPEG 960 px	`video_processor.py:753-780`; `api.py:135-150`	bớt CPU, RAM clip từ ~25 MB → ~6 MB / camera
Q9	MJPEG chờ `asyncio.Event` / condition khi có frame mới thay vì poll	`api.py:1259-1269`	bớt 0–200 ms jitter
Q10	Reader: số thứ tự frame, xoá frame khi đọc lỗi, timeout "hình cũ" theo thời gian, reconnect có backoff	`video_processor.py:189-247`	hết đứng ở frame cũ; báo mất tín hiệu đúng
Q11	Đo đạc: log/expose `capture_fps, infer_ms (p50/p95), publish_fps, frame_age_ms` mỗi camera trên `/api/v1/system/health`	`api.py:1593`	có số để nghiệm thu

5.2 Tái kiến trúc (P1–P2)

1. **AI worker là process riêng** (multiprocessing hoặc service riêng) để tránh GIL với API/stream; giao tiếp qua shared memory / ZeroMQ / Redis.
2. **Runtime theo phần cứng**: máy thật CPU-only dùng OpenVINO, INT8 khi CPU có VNNI/AMX (mục 5.3–5.4); máy có GPU NVIDIA dùng TensorRT. Gộp frame của nhiều camera vào một lần chạy.
3. **go2rtc** cho video + **overlay canvas** phía client (mục 4).

4. **Lưu trữ**: JSON → SQLite (WAL) cho events/logs/schedules; clip lấy từ ghi segment.

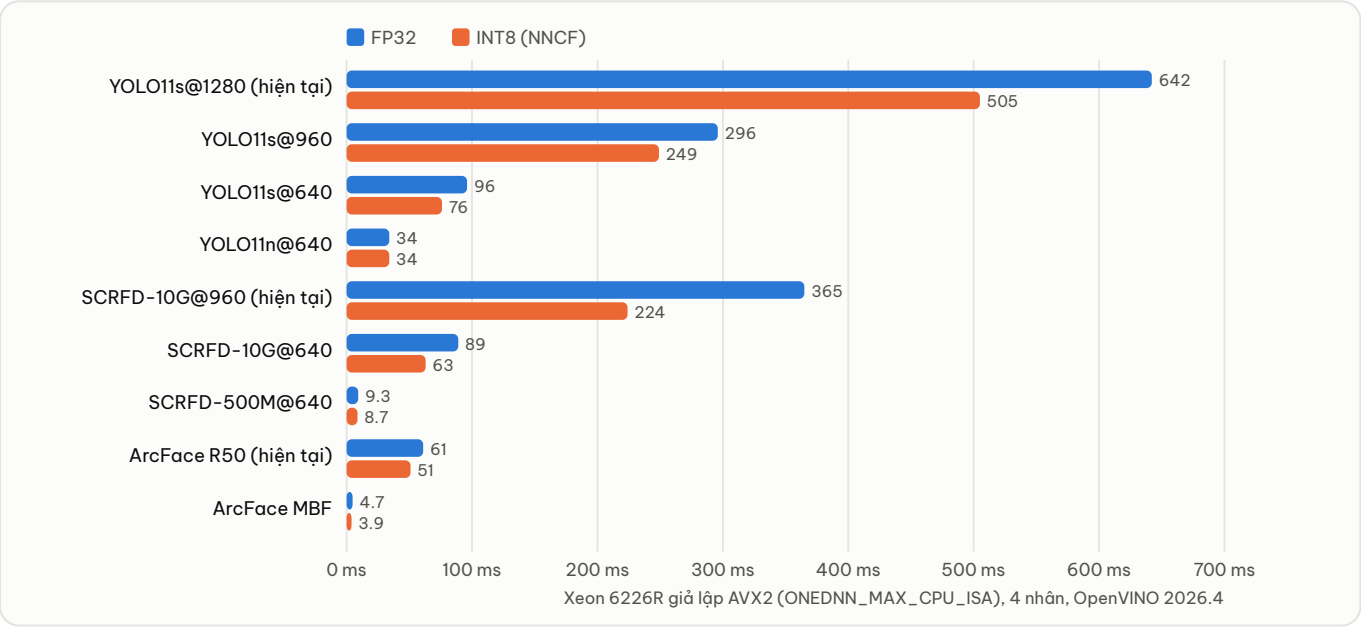
5.3 Hôm nay chạy CPU, sau này thêm GPU: phương án tự thích ứng phần cứng

> **Bối cảnh**
> Máy production hiện **chỉ có CPU**, chưa biết kiến trúc, số camera chưa chốt. **Sau này có thể lắp GPU**, nhiều khả năng là NVIDIA L4. Máy dev `techainer-rtx3090-1` đang chạy bản demo và có GPU, nhưng không đại diện cho production. Yêu cầu: **một codebase tự thích ứng**. Hôm nay tối ưu cho CPU; khi có GPU thì tự nhận và mở rộng (mục 5.5), không phải viết lại. Mọi phần "GPU / TensorRT / NVDEC" ở các mục trên là nhánh GPU của cùng thiết kế này.

Số đo thật trên CPU (02/11/2026)

Môi trường đo:

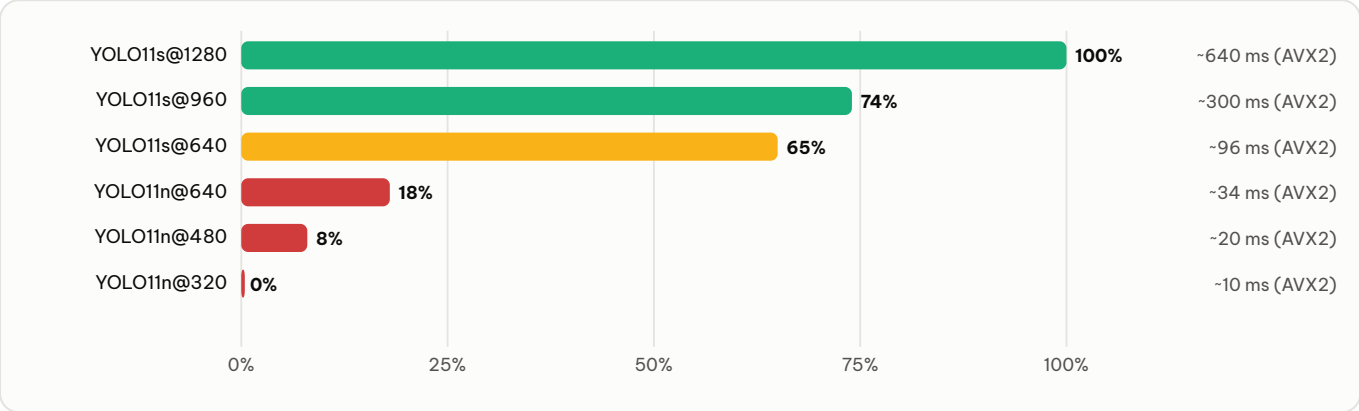
- Máy: Xeon Gold 6226R, ghim 4–8 nhân vật lý. Máy đang có tải nền (load ~22–31) nên số đo dao động khoảng ±30%; mỗi ô lấy best-of-3.
- Phần mềm: OpenVINO 2026.4, NNCF 3.4 (INT8 lượng tử hoá sau huấn luyện, hiệu chỉnh bằng 138 frame camera thật).
- Giả lập CPU đời cũ (AVX2, giống i7-9700K của máy dev) bằng `ONEDNN_MAX_CPU_ISA=AVX2`.
- Giải mã và mã hoá đo bằng ffmpeg 1 luồng, trên clip 20 s ghi từ `cam_01`.



Hình 5.1 — Độ trễ mỗi lần chạy model trên CPU ở mức AVX2, 4 luồng. Cấu hình hiện tại (YOLO11s@1280 + SCRFD-10G@960, cộng tối đa 4 lần quét lại, cộng R50 cho từng mặt) cộng lại khoảng **2–3 s/frame**, khớp với 2–4 s/vòng đo trên server.

CÂU HỎI	KẾT QUẢ ĐO	KẾT LUẬN
FP8?	CPU không có phép tính FP8. OpenVINO trên CPU hỗ trợ FP32, BF16/FP16 (chỉ khi có AMX/AVX512-FP16) và INT8.	Không dùng
INT8 nhanh hơn bao nhiêu?	**CPU có VNNI** (AVX512-VNNI / AVX-VNNI / AMX): model lớn nhanh 1.6–3×, ví dụ SCRFD-10G 101→33 ms, R50 46→15 ms, YOLO11s@640 89→54 ms. **CPU chỉ có AVX2**: chỉ nhanh 1.0–1.6×; model nhỏ (YOLO11n, SCRFD-500M, MBF) gần như không đổi.	Tuỳ CPU: bật khi máy đích có VNNI/AMX **và** qua kiểm độ chính xác
INT8 có làm sai lệch kết quả?	YOLO PTQ cấu hình mặc định: INT8 chỉ bắt lại 89–92% hộp của FP32, và YOLO11s INT8 sinh thêm 15% hộp. ArcFace: cosine giữa embedding FP32 và INT8 là 0.970 (R50), 0.988 (MBF).	Lượng tử hoá YOLO phải bỏ qua phần đầu ra (head), theo cách Ultralytics export. ArcFace INT8 lệch khoảng 0.03, xấp xỉ biên 0.05 của luật khớp, nên phải đo lại trên ảnh đăng ký thật trước khi dùng.

CÂU HỎI	KẾT QUẢ ĐO	KẾT LUẬN
OpenVINO hay ONNX Runtime?	OpenVINO nhanh hơn ở đa số model (R50 32 so với 97 ms, YOLO11n@640 24 so với 43 ms), nhưng trong lần đo này ORT nhanh hơn ở YOLO11s@640 (75 so với 90 ms).	Không chốt cứng: **đo trên máy đích và chọn theo từng model**
Hạ xuống model nhỏ được không?	Xem hình 5.2: cảnh góc rộng, người nhỏ, YOLO11n@640 chỉ còn bắt **18%** số người.	**Không hạ bừa**: giữ độ phân giải ở vùng cần giám sát (cắt ROI), tiết kiệm bằng cách khác



Hình 5.2 — Tỉ lệ hộp người còn bắt được so với YOLO11s@1280 (tham chiếu, không phải nhãn thật), trên 37 frame ban ngày của `cam_01`. Cột phải là độ trễ ở mức AVX2.

Giải mã và mã hoá video trên CPU (1 luồng)

VIỆC	SỐ NHÂN / CAMERA	GHI CHÚ
Chuyển tiếp luồng (go2rtc, `-c copy`)	~0	Cách xem live chính
Giải mã H.265 1080p25 → BGR	0.17	Clip ban đêm, bitrate 0.68 Mbps. Ban ngày có thể gấp 2–3 lần.
Giải mã H.265 360p25 → BGR	0.018	Luồng phụ cho AI, rẻ hơn khoảng 10 lần
Chỉ giải mã keyframe (1080p, GOP 50)	0.015	Chế độ tối thiểu khi máy quá tải (~0.5 fps)
Chuyển mã H.265 → H.264 1080p25 (x264 ultrafast)	0.37	Chỉ dùng khi trình duyệt không xem được HEVC và camera không xuất được H.264
Chuyển mã 360p25	0.04	Fallback rẻ khi xem nhiều ô camera nhỏ
Bộ mã hoá JPEG hiện tại mỗi frame (4 lần mã hoá + base64)	20 ms/frame	Bỏ đi khi overlay chuyển sang vẽ ở trình duyệt

5.4 Thiết kế 3 lớp tự thích ứng

LỚP	KHI NÀO	LÀM GÌ
1. Hồ sơ phần cứng	Lúc cài đặt, hoặc khi phát hiện CPU đổi	`scripts/bench_hw.py` (đã viết; nhánh CPU đã chạy thử, nhánh GPU chưa được chạy trên GPU thật): • Dò CPU (model, số nhân thật tôn trọng taskset/cgroup, cờ AVX2 / AVX512-VNNI / AVX-VNNI / AMX), GPU NVIDIA (`nvidia-smi`, ONNX Runtime CUDA/TensorRT), GPU Intel (OpenVINO GPU), cùng giải mã và mã hoá phần cứng (NVDEC/NVENC, QSV, VAAPI). • Đo từng model trên **mọi backend có sẵn**: OpenVINO CPU FP32/INT8, ORT CPU, ORT CUDA, TensorRT FP16, OpenVINO iGPU FP16. Đo giải mã luồng mẫu bằng phần mềm và bằng phần cứng. • Với từng tầng, thử mọi cách gán thành phần vào backend, có tính ngân sách riêng cho CPU và GPU, rồi chọn cách cho nhiều camera nhất. Ví dụ ở lần đo này, tầng C tự chọn ORT cho phát hiện người và mặt, OpenVINO cho ArcFace. • Tính số nhân cần cho mỗi camera theo từng tầng cấu hình, rồi ra **số camera tối đa**. • Ghi `data/hw_profile.json` kèm **chữ ký phần cứng**. App đọc file này khi khởi động; nếu chữ ký đổi (ví dụ vừa cắm GPU) thì tự đo lại rồi chọn tầng mới.
2. Hồ sơ từng camera	Khi thêm camera, và tự kiểm định kỳ	• Dò các luồng (ONVIF/ffprobe: codec, độ phân giải, fps). • Luồng xem: ưu tiên H.264 để go2rtc chuyển thẳng. Nếu chỉ có HEVC thì phát HEVC cho trình duyệt hỗ trợ, chuyển mã 360p cho trình duyệt còn lại. Giải mã bằng phần cứng khi có. • Luồng AI: chọn luồng nhỏ nhất vẫn đủ **cỡ người** (đo trung vị chiều cao hộp người trong vài phút đầu). • Chỉ chạy AI trên **vùng ROI** (bao của các zone), không chạy cả khung hình. Nếu ROI vẫn lớn thì chia ô (tiling). • **Tự kiểm chất lượng**: cứ 5 phút chạy model tham chiếu (YOLO11s@1280) trên 1 frame và so với cấu hình đang chạy. Còn bắt được dưới 90% số người thì nâng cấp; trên 98% liên tục 1 giờ thì thử hạ một bậc.
3. Điều khiển lúc chạy	Mỗi giây	• Bộ lập lịch chung có **ngân sách nhân CPU** (lấy từ lớp 1). • **Chỉ chạy AI khi có chuyển động**: so khác biệt frame trên ảnh thu nhỏ 160 px trong ROI, tốn khoảng 0.1 ms. Vùng tĩnh thì 1 fps; có chuyển động thì nâng lên fps của tầng. • **Thứ tự ưu tiên**: vùng cấm đang có buổi bắn → cửa sổ điểm danh đầu/cuối buổi → camera có chuyển động → camera tĩnh. • Nhận diện mặt có hạn mức riêng: chỉ cho track chưa có danh tính, ưu tiên mặt to. • Khi frame chờ quá 1 s, hạ dần theo thứ tự: giảm fps → giảm hạn mức nhận diện mặt → hạ tầng model. Chỉ nâng lại sau 60 s ổn định, để không nhảy qua lại. • Thêm camera vượt năng lực thì **báo ngay trên UI**, không để cả hệ thống chậm đi mà không ai biết.

Thang cấu hình và năng lực ước tính khi chỉ có CPU (8 nhân AVX2, tương đương i7-9700K)

TẦNG	NGƯỜI	TÌM MẶT	NHẬN DIỆN MẶT	FPS KHI CÓ CHUYỂN ĐỘNG	CAMERA TỐI ĐA (LUÔN CHUYỂN ĐỘNG)	CAMERA TỐI ĐA (CHUYỂN ĐỘNG 30%)
S (dành cho GPU)	YOLO11s@1280	SCRFD-10G@960	R50	10	0	0
A	YOLO11s@960	SCRFD-10G@640	R50	5	0	1–2
B	YOLO11s@640 (ROI)	SCRFD-10G@640	MBF	5	1	3–4

TẦNG	NGƯỜI	TÌM MẶT	NHẬN DIỆN MẶT	FPS KHI CÓ CHUYỂN ĐỘNG	CAMERA TỐI ĐA (LUÔN CHUYỂN ĐỘNG)	CAMERA TỐI ĐA (CHUYỂN ĐỘNG 30%)
C	YOLO11s@640 (ROI)	SCRFD-500M@640	MBF	3	3	6–8
D (chỉ camera gần)	YOLO11n@640	SCRFD-500M@640	MBF	3	8	11–15

Số liệu do `bench_hw.py` tính:

- Đo trên 8 nhân Xeon 2.9 GHz đang có tải nền, giả lập AVX2. Cột "chuyển động 30%" ghi khoảng giá trị từ 2 lần chạy, vì máy đo dao động.
- Ngân sách là (số nhân − 1.5) × 70%.
- i7-9700K có xung nhịp cao hơn nên có thể được hơn khoảng 1.3–1.5 lần.
- CPU có VNNI/AMX cộng INT8 thì tầng A/B rẻ đi 2–3 lần.
- Đây là trần trên ước tính tuyến tính. **Phải chạy script trên máy đích** để chốt.

```
# Chạy trên máy đích (không cần torch)
uv run --with openvino --with nncf python scripts/bench_hw.py \
    --models models/ --cameras 4 --duty 0.3 --clip luong_phu_360p.mp4 --out data/hw_profile.json
# Máy có GPU NVIDIA: thêm --with onnxruntime-gpu (và tensorrt-cu12 để đo TensorRT)
# --near: cho phép tầng D khi camera đặt gần; --duty: tỉ lệ thời gian vùng có chuyển động; --cpu-only: bỏ qua GPU
```

Việc cần làm cho nhánh CPU (làm ngay)

1. Đổi runtime sang **OpenVINO** (export YOLO `format=openvino`; SCRFD/ArcFace đọc ONNX bằng OpenVINO). Giữ ORT làm phương án dự phòng; chọn runtime theo số đo trong `hw_profile`.
2. Gói sẵn bộ IR INT8 đã kiểm độ chính xác: YOLO lượng tử hoá bỏ qua phần head; ArcFace thì đo TAR/FAR trên ảnh đăng ký thật. App chỉ bật INT8 khi CPU có VNNI/AMX.
3. Chạy AI trên ROI hoặc chia ô, chỉ chạy khi có chuyển động, và có bộ lập lịch theo ngân sách CPU (lớp 3).
4. go2rtc chuyển thẳng luồng xem; chỉnh camera sang H.264 nếu được. AI đọc luồng phụ.
5. Ghim số luồng, ví dụ `INFERENCE_NUM_THREADS` = 2–4 cho mỗi worker, và ghim worker AI vào các nhân riêng (cpuset). Đo trên Xeon cho thấy 2→4 luồng chỉ nhanh thêm 1.2–1.9 lần, nên nhiều worker 2 luồng hiệu quả hơn một worker dùng hết nhân.
6. Theo dõi trên `/api/v1/system/health`: tầng đang chạy, fps AI, tuổi frame, ngân sách CPU còn lại.

5.5 Khi có GPU (dự kiến NVIDIA L4): mở rộng trên cùng codebase

Mọi thứ đi qua một lớp backend chung, kiểu `Detector.infer(batch)`, có 3 cách cài: OpenVINO, ONNX Runtime, TensorRT. Model gốc chỉ có một bộ ONNX; artifact riêng cho từng backend (IR INT8, engine TensorRT) được build lúc cài đặt và cache theo chữ ký phần cứng. Khi có GPU, chỉ đổi cấu hình, không đổi code nghiệp vụ.

THÀNH PHẦN	HÔM NAY (CPU)	KHI CÓ GPU NVIDIA (L4)
Runtime model	OpenVINO / ORT CPU, chọn theo số đo; INT8 chỉ khi CPU có VNNI/AMX	TensorRT FP16 (qua ORT TensorRT EP hoặc engine riêng); INT8 TensorRT sau khi hiệu chỉnh và kiểm độ chính xác
FP8	Không có trên CPU	L4 (kiến trúc Ada) có FP8 trong TensorRT, nhưng lợi chủ yếu cho transformer. Với CNN (YOLO, SCRFD, ArcFace) thì FP16/INT8 đã đủ; chỉ dùng FP8 nếu đo ra nhanh hơn mà không giảm độ chính xác
Tầng cấu hình	B/C: YOLO11s@640 trên ROI, 3–5 fps khi có chuyển động	S/A: YOLO11s@960–1280 (hoặc lớn hơn), 10 fps; nhận diện mặt bằng R50

THÀNH PHẦN	HÔM NAY (CPU)	KHI CÓ GPU NVIDIA (L4)
Gom batch	Không cần (CPU ít lợi)	Gom frame của nhiều camera vào một lần chạy để tận dụng GPU
Giải mã luồng AI	CPU, luồng phụ 360p/720p	NVDEC (L4 có 4 bộ giải mã), giải mã thẳng vào bộ nhớ GPU, đọc được luồng chính độ phân giải cao
Luồng xem	go2rtc chuyển tiếp; nếu phải chuyển mã thì libx264 360p	Vẫn chuyển tiếp; nếu phải chuyển mã thì dùng NVENC, đủ cả 1080p
Ngân sách	Nhân CPU	% GPU + nhân CPU cho tiền xử lý và hậu xử lý; script tính cả hai
Khi GPU lỗi hoặc bận	—	Tự lùi về tầng CPU và báo trên UI; vẫn giữ giám sát vùng cấm

Chưa có số đo GPU cho production: chưa có máy L4, và không dùng số của một GPU khác để suy ra. Khi có máy L4 thì chạy `bench_hw.py` để có số thật. Muốn kiểm nhánh GPU sớm thì có thể chạy trên `techainer-rtx3090-1` (Ampere, cùng hệ CUDA/TensorRT), nhưng máy này đang chạy app và job train, nên cần lịch chạy riêng.

6. Đánh giá AI

THÀNH PHẦN	HIỆN TRẠNG	RỦI RO	ĐỀ XUẤT
Phát hiện người	YOLO11s COCO, chỉ class person, conf 0.25, imgsz 1280 (`detector.py`)	Tốt cho người nhỏ/xa nhưng đắt. Hộp "giữ lại khi bị che" 5 s vẫn được tính vào quân số và **có thể kích hoạt xâm nhập** (`video_processor.py:552`).	Giữ YOLO11s, chạy trên ROI hoặc chia ô (YOLO11n mất phần lớn người, hình 5.2); OpenVINO trên CPU; loại hộp held khỏi luật xâm nhập.
Tracking	ByteTrack cấu hình mặc định (`frame_rate=30`, `track_buffer=30`)	Chạy <1 fps ⇒ người di chuyển xa giữa 2 frame, đổi ID liên tục.	Đặt `frame_rate` = fps AI thật; sau khi tách pipeline chạy AI ≥10 fps.
Nhận diện mặt	buffalo_l: SCRFD-10G + ArcFace R50 (comment ghi nhầm "ResNet100", `face_engine.py:30`); cosine, ngưỡng mặc định 0.36, **deploy đặt 0.30**; yêu cầu cách người thứ 2 ≥ 0.05	0.30 thấp cho ArcFace trên ảnh CCTV ⇒ dễ nhận nhầm khi danh sách lớn. Gán mặt → người kiểu tham lam theo thứ tự.	Hiệu chỉnh ngưỡng trên dữ liệu thật (mục tiêu FAR ≤ 0.1%), thường rơi 0.40–0.50; gán Hungarian theo độ tương đồng.
Đăng ký mặt	1 ảnh/người, lấy mặt lớn nhất, không kiểm chất lượng; đăng ký lại thì thay thế	Ảnh chính diện khớp kém với góc CCTV từ trên cao.	3–5 ảnh nhiều góc, gallery/trung bình embedding; chặn ảnh mờ, nhỏ, nghiêng (det_score, kích thước, độ nét, yaw).
Gắn danh tính vào track	Cộng dồn điểm, bind ở 0.7 (~2 lần khớp), TTL 20 s, không bao giờ gỡ	Sau khi ByteTrack đổi ID, **người sai vẫn được ghi có mặt**.	Gỡ/đổi danh tính khi có khớp mạnh trái chiều; ghi log xung đột.
Chống giả mạo	Không có	Giơ ảnh/điện thoại có mặt người khác là điểm danh hộ được.	Thêm anti-spoof (MiniFASNet/Silent-Face) cho các lần khớp dùng để điểm danh.
Điểm danh / presence	`MIN_SIGHTINGS=3` tính theo **số frame** (`presence.py:15`, `attendance.py:19`); xử lý ca qua đêm đúng	Frame trùng lặp (reader trả lại frame cũ) hoặc vài frame nhận nhầm liên tiếp ⇒ có mặt giả.	≥3 lần thấy trên ≥2 s và frame khác nhau.
An toàn (vùng cấm, vạch)	Điểm chân trong polygon; đổi dấu tích chéo cho vạch; cooldown 20 s/zone; `dwell_seconds` có tính nhưng không dùng làm điều kiện; `detect_object` lưu nhưng không dùng	Nhiễu chân hộp sát biên, hộp held, đổi ID ⇒ báo động đỏ ngay frame đầu. Log có **157 INTRUSION** trong một buổi.	Dwell tối thiểu 1–2 s hoặc N frame liên tiếp; vùng đệm hysteresis; lọc hướng cho vạch; áp dụng `detect_object` hoặc bỏ khỏi UI.
Giám sát quân số	Trung vị 5 lần đếm, cảnh báo sau 30 s, cooldown 60 s	Thông điệp dùng số đếm thô thay vì số đã làm mượt (`video_processor.py:793`).	Dùng số làm mượt trong thông điệp.
Nhiều camera	1 YOLO / camera; chung 1 event loop	Tải tăng tuyến tính, camera này làm chậm camera kia.	Model dùng chung + batch, worker riêng.

Thiếu bộ đánh giá AI

Hiện không có cách đo độ chính xác khi đổi ngưỡng/model. Đề xuất: quay 3–5 clip thật tại thao trường (sáng/chiều, đông/thưa, đi ngang vùng cấm), gán nhãn (người, ID, danh tính, thời điểm vào vùng), viết script chạy offline ra các chỉ số: **precision/recall phát hiện người**, **IDF1** cho tracking, **TAR@FAR=0.1%** cho nhận diện mặt, **tỉ lệ báo động giả/giờ** cho xâm nhập, **độ chính xác điểm danh** theo buổi. Chạy trong CI mỗi khi đổi tham số.

7. Ma trận yêu cầu (FunctionList v0.3)



Hình 7.1 — Độ phủ theo sheet, đếm theo từng dòng giao dịch. Sheet không có cột ưu tiên hay tiêu chí nghiệm thu, nên chấm theo câu chữ của từng dòng.

7.1 Tiêu chí chấm

- Xong: người dùng làm được trọn thao tác mà dòng yêu cầu, trên UI, với dữ liệu thật từ backend, và có bằng chứng trong code. Đây là **"xong về chức năng"**. Không bao gồm:
  - **Phân quyền.** Mọi dòng đều ghi actor (CBQH hoặc QTHT), nhưng backend không kiểm role (`app/auth.py:1-10`). Role chỉ quyết định menu nào hiện trên UI. Vì vậy cột "Phân quyền" là **Chưa đạt cho cả 28 dòng**. Nặng nhất ở phân hệ III: thao tác của QTHT gọi được thẳng qua API mà không cần đăng nhập.
  - **Chất lượng.** Một dòng "Xong" vẫn có thể có lỗi. Ví dụ R17: luật vùng cấm chạy được nhưng chưa có dwell hay hysteresis, và log một buổi có 157 cảnh báo xâm nhập (mục 6). Những dòng như vậy có đánh dấu ⚠ kèm ghi chú.
- Một phần: có một phần hành vi, hoặc chỉ đạt theo một cách hiểu hẹp, hoặc thiếu một thành phần mà câu chữ yêu cầu rõ (ví dụ "khu vực" là nhóm camera, "giám sát tự động theo cấu hình").
- Chưa có: không có code liên quan.
- Cần KH xác nhận: câu chữ hiểu được theo 2 cách và kết quả chấm phụ thuộc vào cách hiểu.

Toàn bộ trích dẫn `file:line` theo commit `6fc2929`. Mã app giống hệt bản đang deploy `ac6f28b`: `git diff --stat ac6f28b 6fc2929` chỉ đổi `.dockerignore`, `.env.example`, `Dockerfile`, `docker-compose*.yml` và `scripts/probe_ai_camera.py`. Cột TT ghi theo UC trong sheet (UC1-1 là dòng giao dịch thứ 1 của UC 1). Cột "Dòng" là số dòng trong Excel.

7.2 Sheet "AI Giám sát": 28 dòng

Phân quyền: Chưa đạt cho cả 28 dòng (xem mục 7.1). Cột Actor ghi vai trò mà sheet yêu cầu.

TT	DÒNG	ACTOR	YÊU CẦU (TRÍCH)	TRẠNG THÁI	BẰNG CHỨNG / CÒN THIẾU
I. Tổ chức thực hiện HL đào tạo, UC 1: Theo dõi tiến độ HL đào tạo					
UC1-1	R5	CBQH	Tổng hợp thông tin lịch HL đào tạo	Xong	`#view-schedule-progress`; `GET /api/v1/summary/training` (`api.py:1034`) lọc theo loại, ngày, ca, trạng thái; `loadTrainingSchedule` (`app.js:2453`)
UC1-2	R6	CBQH	Theo dõi chi tiết lịch HL	Xong	`#view-session-detail`; `openSessionDetail` (`app.js:2524`)
UC1-3	R7	CBQH	Theo dõi tiến độ HL hiện tại	Xong	`time_progress_pct`; thanh tiến độ ở `app.js:2486`
UC1-4	R8	CBQH	Theo dõi tiến độ **hoàn thành** HL	Một phần	Chỉ có tiến độ theo từng buổi. Thẻ tổng đã bị bỏ có chủ đích. `overall_progress_pct` vẫn được tính (`api.py:1147`) nhưng `app.js` không dùng. Chưa có tiến độ theo chương trình hay kế hoạch.
UC 2: Giám sát quân số thao trường HL đào tạo					
UC2-1	R10	CBQH	Màn tổng hợp giám sát quân số HL đào tạo	Một phần	Không có màn riêng. Cột "Quân số" chỉ hiện số yêu cầu. `present_start` và `present_end` có trong API (`api.py:1120`) nhưng `app.js` không dùng.

TT	DÒNG	ACTOR	YÊU CẦU (TRÍCH)	TRẠNG THÁI	BẰNG CHỨNG / CÒN THIẾU
UC2-2	R11	CBQH	Kết quả tham gia theo lịch: không tham gia / chậm / "chưa hết giờ đã kết thúc HL"	Xong Cần KH xác nhận	Chấm theo **cách hiểu cấp cá nhân**: vắng, đến muộn, về sớm của từng người (`presence.py:19, 120`; sự kiện LATE ở `attendance.py:566`, EARLY_LEAVE ở `attendance.py:592`; danh sách theo người qua `GET /api/v1/sessions/{id}/attendance`). Nếu "chưa hết giờ đã kết thúc" nghĩa là **cả buổi hoặc cả đơn vị kết thúc trước giờ** thì code chưa có, và dòng này chỉ là Một phần. Lưu ý: "không tham gia" chỉ hiện trong danh sách điểm danh, **không phát sự kiện ABSENT theo người**. `TYPE_ABSENT` ở `video_processor.py:791` là cảnh báo quân số tụt dưới sĩ số chuẩn, không phải vắng theo người.
UC2-3	R12	CBQH	Quân số đầu buổi và cuối buổi	Xong	`#sd-checks-tbody`; `GET /api/v1/sessions/{id}/checks` (`api.py:1531`)
UC2-4	R13	CBQH	Hình ảnh điểm danh đầu và cuối buổi	Xong	`_save_evidence` (`attendance.py:416`); thumbnail trong `renderSessionChecks` (`app.js:2630`)
UC 3: Giám sát an toàn HL đào tạo bắn đạn thật					
UC3-1	R15	CBQH	Màn giám sát an toàn bắn đạn thật	Xong	`#view-safety`; `GET /api/v1/summary/safety` (`api.py:918`)
UC3-2	R16	CBQH	Phát hiện đối tượng vào **khu vực trường bắn** khi đang HL	Một phần	Có thể vẽ một `restricted_area` bao quanh cả trường bắn, nên phần phát hiện kỹ thuật là có. Còn thiếu 2 thứ: • Không có **loại vùng riêng "khu vực trường bắn"**. Hiện chỉ có `restricted_area` và `crossing_line` (`safety.py:20-21`), nên không tách được với "vùng cấm (khối chắn)" khi hiển thị và thống kê. • Luật **không gắn với buổi bắn theo lịch**: chạy 24/7, không kiểm có buổi đang diễn ra (`video_processor.py:552`). Ngoài ra chỉ phát hiện người: `detect_object` được lưu nhưng không dùng.
UC3-3	R17	CBQH	Phát hiện đối tượng vào **vùng cấm (khối chắn)**	Xong ⚠	`RULE_RESTRICTED` và `_inside_polygon` (`safety.py:20, 240`); banner đỏ và nút xác nhận xử lý. ⚠ Chất lượng: không có dwell hay hysteresis, hộp "giữ lại khi bị che" vẫn kích hoạt được luật; 157 cảnh báo trong một buổi (mục 6).
UC3-4	R18	CBQH	Hình ảnh vi phạm an toàn	Xong	Ảnh chụp và clip mp4 10 s (`GET /api/v1/events/{id}/clip`, `api.py:839`)
II. Quản lý giám sát HL chiến đấu, UC 4: Giám sát quân số thao trường HLCĐ					
UC4-1	R21	CBQH	Màn tổng hợp giám sát quân số HLCĐ	Một phần	Giống UC2-1
UC4-2	R22	CBQH	Không tham gia / chậm / "chưa hết giờ đã kết thúc" HLCĐ	Xong Cần KH xác nhận	Giống UC2-2, cùng cách hiểu cấp cá nhân
UC4-3	R23	CBQH	Quân số đầu buổi và cuối buổi HLCĐ	Xong	Giống UC2-3
UC4-4	R24	CBQH	Hình ảnh điểm danh HLCĐ	Xong	Giống UC2-4
UC 5: Giám sát an toàn HLCĐ bắn đạn thật					

TT	DÒNG	ACTOR	YÊU CẦU (TRÍCH)	TRẠNG THÁI	BẰNG CHỨNG / CÒN THIẾU
UC5-1	R26	CBQH	Màn giám sát an toàn HLCĐ	Xong ⚠	Dùng chung màn với UC3-1, lọc bằng nút `chien_dau` (`index.html:719-722`; `setTrainingFilter` ở `app.js:2414`; lọc camera ở `app.js:2771`). ⚠ Rủi ro phân loại: xem mục 7.3.
UC5-2	R27	CBQH	Vào khu vực trường bắn HLCĐ	Một phần	Giống UC3-2
UC5-3	R28	CBQH	Vào vùng cấm (khối chắn) HLCĐ	Xong ⚠	Giống UC3-3
UC5-4	R29	CBQH	Hình ảnh vi phạm HLCĐ	Xong	Giống UC3-4
III. Phân hệ Quản trị giám sát khu vực, UC 6: Cấu hình khung giờ và quy tắc kích hoạt tự động					
UC6-1	R32	QTHT	Xem cấu hình khung giờ và quy tắc kích hoạt	Xong	`#view-schedule`; `GET /api/v1/schedules` (`api.py:1468`)
UC6-2	R33	QTHT	Cập nhật cấu hình và **giám sát tự động theo cấu hình đã lưu**	Một phần	CRUD lịch đủ (`api.py:1482/1501/1521`). Thiếu phần tự động: • Điểm danh chỉ tự mở **khi camera đã đang chạy** (`maybe_open_scheduled`, `attendance.py:662`). Không có cơ chế tự bật hay tắt camera theo lịch. • Cờ `enabled` bị bỏ qua: `_load_schedules` (`attendance.py:369-386`) chỉ lọc theo `camera_id` (`:382-383`), không lọc `enabled`. Cờ này chỉ dùng để lọc danh sách ở API (`api.py:1477-1478`) và không có trên UI. • Vùng an toàn không có khung giờ. • Route cũ `POST /api/schedules` (`api.py:402`) không validate.
UC 7: Quản lý khu vực giám sát					
UC7-1	R35	QTHT	Xem danh sách khu vực giám sát → "danh sách camera giám sát khu vực tương ứng"	Một phần	Theo câu chữ, khu vực là **một nhóm camera**. Code không có thực thể khu vực, chỉ có `area_name` dạng chữ trên từng camera. Không có màn danh sách khu vực hay danh sách camera theo khu vực. Sự kiện còn ghi cứng "Thao trường số 1" (`events.py:26-27`).
UC7-2	R36	QTHT	Cập nhật thông tin khu vực giám sát	Một phần	Cùng chuẩn với UC7-1: **zone không phải khu vực**. Zone là polygon hay vạch trên một camera (`PATCH /api/v1/zones/{id}`, `api.py:1201`). Chỉ sửa được khu vực gián tiếp, bằng cách đổi `area_name` trong form camera.
UC7-3	R37	QTHT	Xoá khu vực giám sát	Một phần	Như UC7-2. Chỉ xoá được zone (`DELETE /api/v1/zones/{id}`, `api.py:1227`), không có thao tác xoá khu vực.
UC 8: Quản lý thiết bị camera giám sát khu vực					
UC8-1	R39	QTHT	Danh sách thiết bị camera	Xong	`#cameras-tbody`; `GET /api/v1/cameras` (`api.py:1362`). ⚠ Bảo mật: trả về mật khẩu RTSP (mục 2).
UC8-2	R40	QTHT	Cập nhật thông tin camera, kiểm tra và lưu	Xong	`PATCH /api/v1/cameras/{id}` (`api.py:1389`) validate bằng `CameraPatch`
UC8-3	R41	QTHT	Xoá camera	Xong	`DELETE /api/v1/cameras/{id}` (`api.py:1404`): trả 409 khi camera đang chạy, xoá luôn các zone của camera

Tổng: 19 Xong (3 dòng có ⚠ chất lượng, 2 dòng cần KH xác nhận cách hiểu) · 9 Một phần (R8, R10, R16, R21, R27, R33, R35, R36, R37) · 0 Chưa có. Nếu KH chọn cách hiểu cấp buổi cho R11/R22 thì còn 17 Xong và 11 Một

phần.

7.3 Phân hệ II: phân loại sự kiện theo loại huấn luyện

Phân hệ II không có màn riêng. Nó dùng lại màn của phân hệ I, lọc bằng nút "Huấn luyện chiến đấu":

- Lịch và tiến độ: `index.html:542-545` gửi tham số `training_type` lên API (`app.js:2459`).
- An toàn: `index.html:719-722` lọc danh sách camera ngay trên trình duyệt (`app.js:2771`).

Đã kiểm cách sự kiện an toàn được xếp loại:

- **Bản thân sự kiện xâm nhập không mang `training_type`.** Loại được suy ra lúc hiển thị, từ ca đang gắn với camera, qua `_schedule_of_camera` (`api.py:903-915`): ưu tiên ca đang chạy, nếu không có thì lấy ca có giờ bắt đầu sớm nhất.
- **Camera chưa gắn ca nào** có `training_type = null`, nên bị ẩn ở cả 2 bộ lọc "đào tạo" và "chiến đấu". Sự kiện của camera đó chỉ thấy được ở chế độ "Tất cả".
- **Camera dùng chung cho cả 2 loại ca:** ngoài giờ ca, camera được xếp theo ca bắt đầu sớm nhất. Vì vậy một vi phạm lúc đang tập chiến đấu có thể hiện dưới loại "đào tạo" khi xem lại. Loại hiển thị đổi theo thời điểm xem, không theo thời điểm xảy ra.
- **Đèn trạng thái và banner báo động không theo bộ lọc:** `state` và `active_intrusion` tính trên mọi sự kiện INTRUSION (`api.py:924-934`; `app.js:2741-2749`). Đang ở bộ lọc "chiến đấu" vẫn có thể thấy báo động của camera "đào tạo".

Đề xuất: ghi `training_type` và `schedule_id` vào sự kiện ngay lúc phát; nếu không có ca nào đang chạy thì ghi "ngoài giờ HL". Đèn trạng thái và banner tính theo bộ lọc đang chọn.

7.4 Sheet "Chat Bot": 26 use case, 63 dòng, chưa có code

Trong repo không có code LLM hay RAG, không có endpoint chat, UI chat hay dependency liên quan. Các UC gồm:

- Tra cứu: tham mưu tác chiến, mốc khống chế pháo binh, CTĐ-CTCT, quân khí, bằng lái xe.
- Xe-máy: 9 chức năng tra cứu, kế hoạch công tác xe-máy, hồ sơ đưa đi sửa chữa.
- Huấn luyện: chương trình, kế hoạch, quân số, kết quả, tổng hợp đánh giá, tư liệu, thiết bị phục vụ HL.
- Tài liệu số hoá, vật tư kỹ thuật, quân y (2 UC), cán bộ (2 UC), HL mô phỏng (4 UC).
- Quản trị Chatbot: thống kê đánh giá (biểu đồ, xuất Excel/ảnh) và câu hỏi chưa trả lời được (danh sách, tìm kiếm, chi tiết, xuất Excel).

UC CHAT BOT	DỮ LIỆU SẴN CÓ TRONG HỆ THỐNG NÀY	MỨC ĐÁP ỨNG NẾU LÀM
9–10: Chương trình và kế hoạch HL	`schedules.json`: bài học, loại HL, đơn vị, ca, khoảng ngày, thứ trong tuần	Một phần. Có lịch, không có "chương trình" theo nghĩa nội dung đào tạo.
11: Quân số HL	`attendance_logs.json`: điểm danh theo buổi và theo người	Gần như đủ
12–13: Kết quả và tổng hợp đánh giá HL	`attendance_logs.json`: tỉ lệ có mặt, muộn, về sớm	Một phần. Chỉ có kết quả chuyên cần, không có điểm hay kết quả huấn luyện.
Các UC còn lại (1–8, 14–24)	Không có	Cần nguồn dữ liệu mới
25–26: Quản trị Chatbot	Không có (phát sinh từ chính Chatbot)	Làm cùng Chatbot

Cần xác nhận phạm vi; chưa ước lượng được công

Nếu Chat Bot thuộc sản phẩm hoặc repo khác thì đánh dấu ngoài phạm vi repo này. Nếu thuộc repo này, **hiện chưa ước lượng được công** vì công phụ thuộc vào 3 giả định chưa có câu trả lời:

- Số miền dữ liệu được cấp, có thể lên tới 19 miền tra cứu.
- Dữ liệu nguồn đã số hoá và có cấu trúc chưa, hay phải số hoá hoặc OCR.
- Ràng buộc triển khai: LLM chạy on-prem (cỡ model và GPU) hay được dùng API ngoài.

Có câu trả lời thì mới ước lượng theo từng miền được. Có thể bắt đầu bằng UC 9–13, dùng dữ liệu đã có ở bảng trên.

8. Vận hành, dữ liệu, triển khai

8.1 Hiện trạng trên `techainer-rtx3090-1`

HẠNG MỤC	HIỆN TRẠNG	MỨC
Cách chạy	`uv run main.py` trong tmux của user `martin`, thư mục `/mnt/ssd/martin/project/military-attendance-check`, cổng 8199. Không systemd/Docker, không restart khi chết. Cloudflared log có ~1.450 lần "connection refused" khi app chết (30/10, 01–02/11).	Cao
Phiên bản	Deploy `ac6f28b`; local đã có `6fc2929` (đóng gói Docker CPU/GPU) chưa deploy. Tham số chạy chỉ qua command line (`FACE_SIM_THRESHOLD=0.3`), không có `.env`.	TB
Tài nguyên	8 core / 62 GB RAM / RTX 3090 24 GB. Load ~22, CPU idle 0%. App ~540% CPU, ~2 GB RAM, chỉ giữ 664 MB VRAM, gần như không dùng GPU. Job train PaddleOCR của người khác chiếm 12.7 GB VRAM và nhiều CPU.	Cao
Đĩa	`data/events`: 119.135 file, **166 GB**; mỗi sự kiện ~470 KB ảnh + 9.5–20 MB mp4. Đĩa `/` 91% (còn 42 GB). Không có cơ chế dọn.	Cao
Camera	`cam_1790514775112` (192.168.1.113) không tới được (No route to host) — log lặp "Stream timeout 30000 ms" mỗi 30 s trước khi restart; người dùng bấm stop 6 lần liên tiếp (nút không phản hồi rõ). Cả 2 camera ghi `source_type: "file"` dù là RTSP.	TB
Lưu trữ	Toàn bộ là file JSON đọc/ghi cả file, không khoá; nhiều camera ghi cùng lúc có thể ghi đè nhau; sự kiện giới hạn 500 bản ghi; bộ đếm ID sự kiện reset sau restart (`_57707` → `_5`).	TB
Tunnel	2 container cloudflared (token, QUIC, edge HKG), bản 2026.9.1 đã cũ; thêm frpc là cửa thứ 2.	Thấp
Log	Chỉ còn trong scrollback tmux; FutureWarning numpy in mỗi lần align mặt gây nhiễu.	Thấp

8.2 Đề xuất

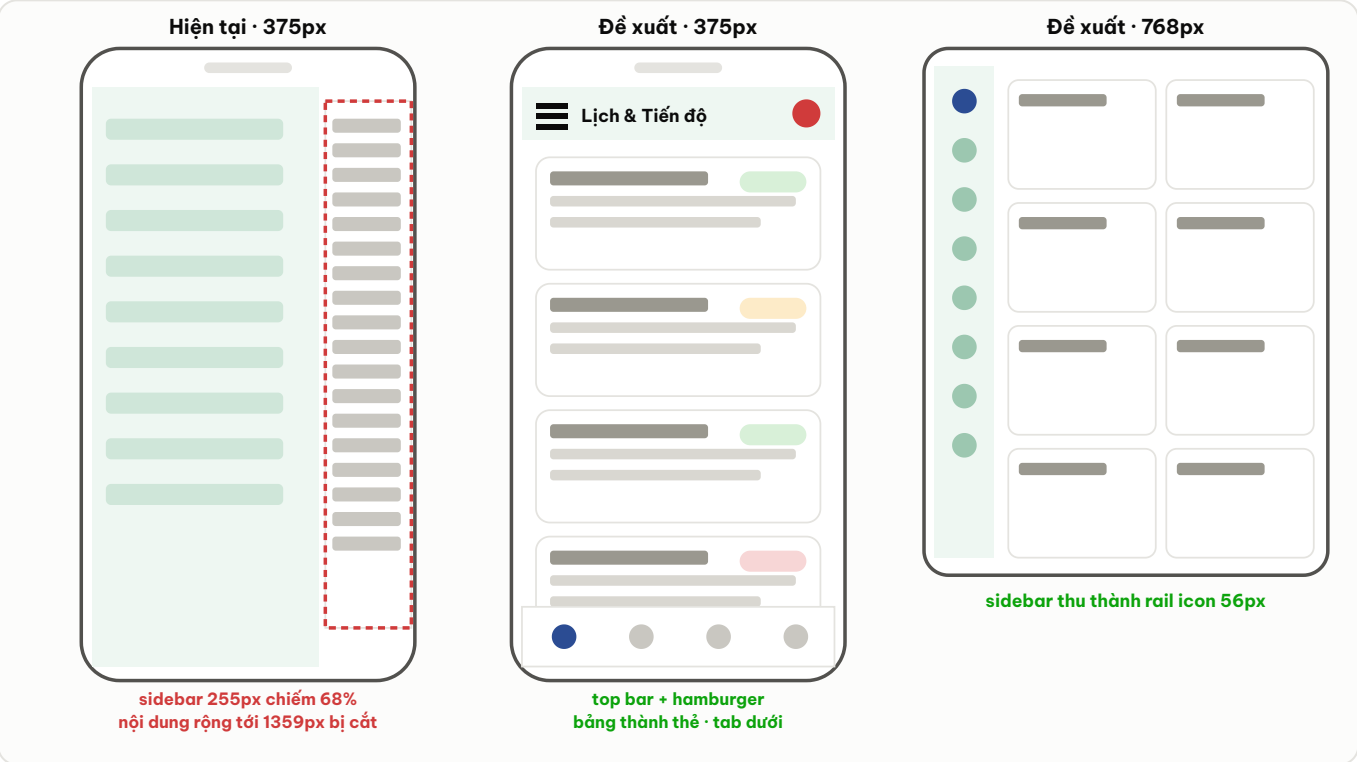
1. **Docker Compose** (đã có ở `6fc2929`) với `restart: unless-stopped`; image mặc định CPU dùng OpenVINO, profile GPU (chỉ máy dev) dùng `onnxruntime-gpu`, `cpus:` / `cpuset` giới hạn, healthcheck gọi `/api/v1/system/health`. Thêm service go2rtc cùng compose.
2. **Cô lập tài nguyên** với job train: ghim CPU (cpuset), chia GPU (hoặc dời job train sang 3090-2).
3. **Retention**: giữ ảnh 30 ngày, clip 7 ngày (tham số hoá), job dọn hằng giờ; cảnh báo khi đĩa >80%. Mp4 ở 960 px/CRF 28 thay vì full-res: giảm ~5–10× dung lượng. Dọn 166 GB hiện có sau khi xác nhận với người dùng.
4. **SQLite (WAL)** cho events/logs/schedules/cameras; ID sự kiện tăng dần bền vững.
5. **Quan sát**: log ra stdout (Docker), metrics fps/độ trễ/frame cũ mỗi camera, cảnh báo khi camera mất tín hiệu >30 s.
6. **Test**: chạy test trên thư mục data tạm (đừng ghi đè `data/*.json` thật), thêm test end-to-end cho vòng video với video mẫu, test chống hồi quy độ trễ (frame_age < ngưỡng).

9. UI/UX & responsive

Đã chụp trang live ở 3 kích thước (375×812, 768×1024, 1440×900) và đo độ rộng nội dung thực tế. Desktop gọn, nhất quán, màu xanh quân sự dễ chịu. Mobile và tablet chưa dùng được.

MÀN HÌNH	375 PX (MOBILE)	768 PX (TABLET)	1440 PX
Lịch & Tiến độ	1140 px	1140 px	vừa
An toàn bắn đạn thật	717 px	vừa	vừa
Nhật ký điểm danh	1216 px	1216 px	vừa
Giám sát trực tiếp	1019 px	1019 px	vừa
Thiết bị camera	737 px	vừa	vừa
Vùng giám sát	1255 px	1255 px	vừa
Đăng ký khuôn mặt	1359 px	1359 px	vừa

Giá trị là `document.scrollWidth`; "vừa" = không vượt màn hình. Phần vượt bị `body{overflow-x:hidden}` (`style.css:43`) cắt mất nên người dùng không cuộn ngang được.



Hình 9.1 — Wireframe hiện tại so với đề xuất. Hiện tại sidebar 255 px chiếm 68% màn hình 375 px, tiêu đề bị bẻ từng chữ; đề xuất: top bar + menu off-canvas, bảng chuyển thành thẻ, tablet dùng rail icon.

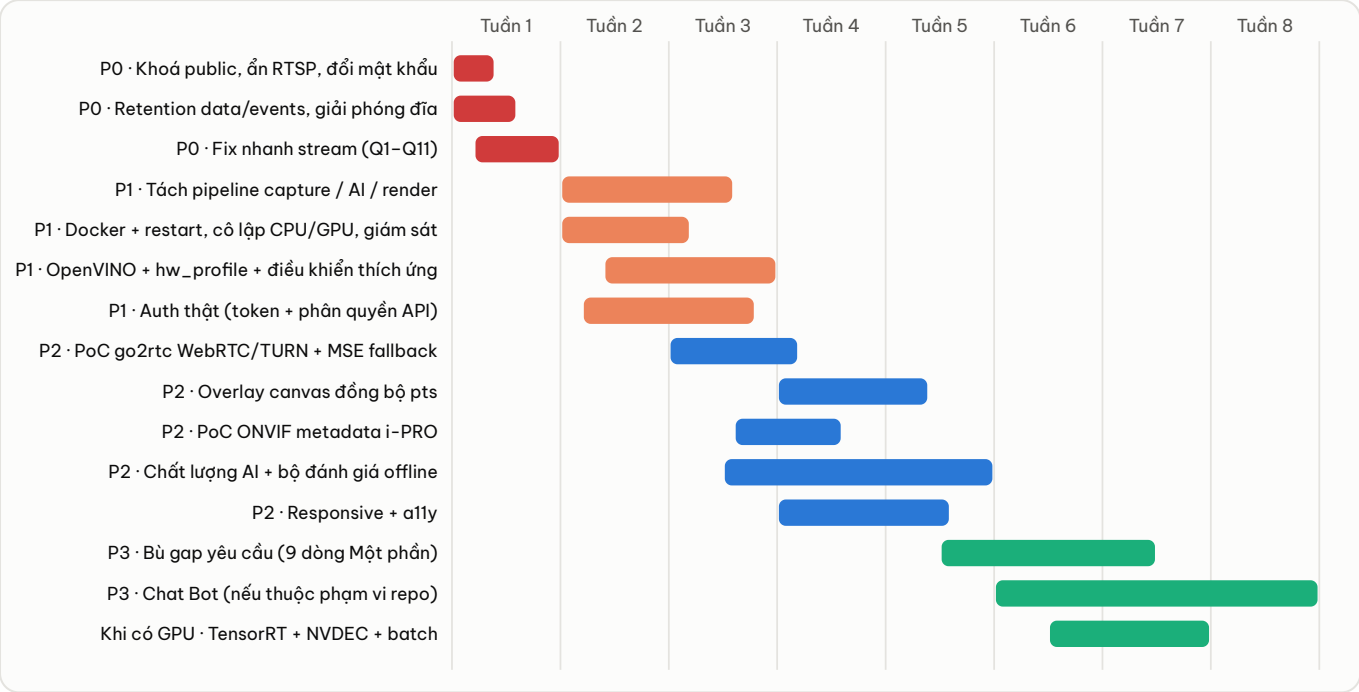
9.1 Vấn đề

#	VẤN ĐỀ	VỊ TRÍ	MỨC
U1	Chỉ 1 breakpoint (`max-width:1180px`); sidebar cố định 255 px, `height:100vh`, không thu gọn	`style.css:53-54, 1933`	Cao
U2	Nút thu gọn sidebar `‹` không có handler	`index.html:79`	TB
U3	Grid cột cứng: `1fr 380px`, `420px 1fr`, form `1fr 1fr`; camera wall `minmax(320px,1fr)`	`style.css:355, 792, 849, 1298, 2165`	Cao
U4	Không có trạng thái stream: không báo "hình cũ X giây", không fps, khi lỗi chỉ xoá `src` → icon ảnh vỡ (thấy ở "Sân 2"); 409 "camera chưa chạy" không giải thích	`app.js:500-508`	Cao
U5	Đồng hồ lệch giả: `/api/time` trả giờ không kèm múi giờ (`2026-11-02T18:08:50`); máy không ở GMT+7 hiện "⚠ Máy trạm lệch 420 phút" và đồng hồ sai 7 giờ. Nên dùng `clock.iso()` (có offset)	`api.py:676`, `app.js:66-83`	TB
U6	Chuông thông báo không có handler; breadcrumb mặc định "Giám sát trực tiếp" dù trang chủ là Lịch & Tiến độ; giờ placeholder cứng "23:18:13 17/08/2026"; reload không giữ tab (không đọc hash)	`index.html:151, 167, 171`; `app.js:146, 3266`	Thấp
U7	Accessibility: 0 thuộc tính `aria-*`, 14/61 label có `for`, 5 `div` bấm được không dùng được bằng bàn phím, không có `:focus-visible`, icon emoji không `aria-hidden`	`index.html`	TB
U8	Tương phản < 4.5:1: chữ trắng trên nút xanh `#0a8f4c` (4.16), badge đỏ (3.76), chữ cam cảnh báo `#d97706` (3.19), subtitle 10 px (3.79); 20 rule cỡ chữ 10–11 px; vùng chạm 26–36 px	`style.css`	Thấp
U9	Lỗi hiển thị bằng `alert()`; trang an toàn chỉ `console.error`; không có skeleton khi tải camera	`app.js:318, 1044, 2752`	Thấp
U10	Chính tả không thống nhất: khoá/khóa, xoá/xóa, hoá/hóa, Hủy; tiêu đề tiếng Anh "HORUS AI WEB · MILITARY EDITION" giữa giao diện tiếng Việt	toàn bộ	Thấp
U11	Polling song song: trạng thái 2 s, an toàn 15 s, lịch 30 s — trùng với SSE đã có	`app.js:789`	Thấp

9.2 Đề xuất

- **Breakpoint**: ≤1280 sidebar thu thành rail icon 56 px (nút `‹` hoạt động, nhớ trạng thái); ≤768 sidebar thành menu off-canvas + top bar có hamburger; ≤480 mọi grid 1 cột, header pills gom vào menu.
- **Bảng**: ≤768 chuyển thành thẻ (mỗi dòng là card, cột quan trọng lên đầu) hoặc ít nhất bỏ `overflow-x:hidden` trên `body` để `.table-responsive` cuộn được.
- **Camera wall**: `grid-template-columns:repeat(auto-fill,minmax(min(100%,320px),1fr))`; ô video giữ `aspect-ratio:16/9`.
- **Trạng thái stream**: badge LIVE (xanh) / TRỄ (cam, hiện tuổi hình) / MẤT TÍN HIỆU (đỏ) dựa trên `frame_age_ms` từ server; placeholder + nút thử lại thay icon ảnh vỡ; nút Dừng/Chạy có trạng thái đang xử lý.
- **A11y**: `label for`, `aria-label` cho nút icon, `button` thay `div` bấm được, `:focus-visible`, nâng màu nút chính lên `#087a41` (≥4.5:1), chữ tối thiểu 12 px, vùng chạm ≥ 40 px.
- **Thông báo**: toast thay `alert()`; chuông mở danh sách sự kiện chưa xử lý (dùng lại SSE).
- Kiểm thử responsive tự động: thêm vào `test_ui.mjs` (hoặc Playwright) kiểm `scrollWidth ≤ innerWidth` ở 375/768/1440 cho mọi view.

10. Lộ trình & tiêu chí nghiệm thu



Hình 10.1 — Lộ trình 8 tuần cho 1–2 dev. Đỏ = P0 khẩn, cam = P1, xanh dương = P2, xanh lá = P3.

PHA	NỘI DUNG	CÔNG (ƯỚC)	TIÊU CHÍ NGHIỆM THU
P0 tuần 1	Cloudflare Access, tắt frpc, che `source_uri`, đổi mật khẩu camera/token; retention + dọn đĩa; fix nhanh Q1–Q11	4–6 ngày	Không endpoint nào truy cập được khi chưa đăng nhập SSO; API không trả mật khẩu; đĩa <70%; stream ≥10 fps, trễ video p95 <1.5 s qua Cloudflare; không đứng hình >2 s khi có sự kiện; log ghi rõ thiết bị AI đang chạy
P1 tuần 2–3	Tách AI worker (process), Docker Compose + restart + cô lập tài nguyên, auth phía server (token + phân quyền), SQLite; OpenVINO + `hw_profile` + điều khiển thích ứng (mục 5.4), chạy `bench_hw.py` trên máy đích	10–14 ngày	Kill app → tự lên <30 s; trên máy đích, AI đạt fps của tầng mà `hw_profile` chọn, với số camera đã cam kết; còn bắt ≥90% số người so với model tham chiếu; 2 role bị chặn đúng ở backend (test); 2 camera ghi đồng thời không mất dữ liệu
P2 tuần 3–5	go2rtc WebRTC/TURN + MSE fallback; overlay canvas đồng bộ pts; PoC ONVIF metadata i-PRO; chất lượng AI + bộ đánh giá offline; responsive + a11y	12–15 ngày	Video p95 <1 s (MSE) / <0.5 s (WebRTC); overlay lệch ≤200 ms; không đứng hình >1 s trong 1 giờ; báo động giả xâm nhập <1/giờ trên clip đánh giá; TAR@FAR 0.1% được báo cáo; `scrollWidth ≤ innerWidth` ở 375/768
P3 tuần 5–8	Gap yêu cầu: tự bật/tắt giám sát theo lịch + cờ `enabled` (R33), thực thể khu vực + CRUD khu vực (R35–R37), luật trường bắn gắn lịch (R16/R27), màn tổng hợp quân số (R10/R21), tiến độ hoàn thành (R8), xác nhận cách hiểu R11/R22 với KH; khi có GPU: build engine TensorRT, NVDEC, gom batch và chạy `bench_hw.py` trên máy GPU (mục 5.5); *để xuất ngoài FunctionList: xuất Excel vi phạm*; Chat Bot nếu thuộc phạm vi	10–15 ngày (Chat Bot: chưa ước lượng được, xem mục 7.4)	28/28 dòng AI Giám sát đạt cả chức năng lẫn phân quyền; demo theo kịch bản từng dòng của FunctionList

11. Phụ lục: số liệu gốc & cách đo

11.1 Đo stream qua Cloudflare (150 s)

```
# Kéo MJPEG, cắt từng JPEG theo marker FFD8/FFD9, ghi giờ nhận (GMT+7)
python3 grab.py "https://military-attendance-dev.techainer.com/api/v1/cameras/cam_01/stream.mjpg?overlay=1" cf 150
# 37 frame, ~102 KB/frame, 0.25 fps; khoảng cách TB 4.13 s (min 2.00, p95 6.22, max 7.27)

# Hiệu chỉnh đồng hồ camera: lấy keyframe trực tiếp trên server, đọc OSD
ffmpeg -rtsp_transport tcp -skip_frame nokey -i "rtsp://***@192.168.1.5:554/Streaming/Channels/101" -frames:v 1 ...
# OSD 17:59:07 lúc server 18:05:48.65 ⇒ camera chậm ~401.6–402.3 s

# Codec
ffprobe ... Channels/101 → hevc Main 1920x1080 25/1
ffprobe ... Channels/102 → hevc Main 640x360 25/1
```

Tuổi hình từng frame (s): 4.8 3.2 4.6 4.0 2.8 2.7 2.5 2.9 3.1 4.2 3.5 4.1 3.7 5.9 2.6 2.2 1.6 1.9 4.5 3.7 3.2 5.4 3.0 2.6 3.1 4.1 1.7 2.7 3.7 6.0 1.0 1.4 0.6 0.6 1.6 1.0 1.6. Một lần đo khác (20 s, 4 luồng song song, cả localhost) cho 0.15 fps, khoảng cách 6.5–7.9 s ± 2.1–2.5 s — tải máy dao động theo job train chạy chung.

11.2 Bằng chứng GPU không được dùng

```
Specified provider 'CUDAExecutionProvider' is not in available provider names.
Available providers: 'AzureExecutionProvider, CPUExecutionProvider'
Applied providers: ['CPUExecutionProvider']   # cả 5 model buffalo_l
nvidia-smi pmon -s u   # app: sm "-" trong 15/15 mẫu; NVDEC 0%
```

11.3 Phạm vi & giới hạn

- Chỉ đọc: không sửa code, không restart, không đổi cấu hình server. Trên server chỉ tạo file tạm trong `/tmp` khi lấy keyframe, đã xoá ngay.
- Phân rã thời gian từng bước trong vòng xử lý (hình 3.5) là **ước tính** từ chu kỳ đo được; cần instrument (Q11) để có số chính xác.
- Camera i-PRO không tới được từ server lúc review, nên chưa kiểm chứng được ONVIF metadata trực tiếp; thông tin lấy từ `docs/camera_ai_stream_guide.md` của dev.
- Mức cải thiện ở mục 5.1–5.2 là kỳ vọng theo kinh nghiệm; mục 5.3–5.4 là số đo thật nhưng trên Xeon 6226R đang có tải nền, mức AVX2 là giả lập, và clip quay ban đêm. Cần chạy `scripts/bench_hw.py` trên máy đích để chốt.
- Sheet Chat Bot được đánh giá là "chưa có" trong repo này; chưa kiểm tra các repo khác.
- Mọi trích dẫn `file:line` theo commit `6fc2929`. Mã app (`app/`, `static/`) giống hệt bản đang deploy `ac6f28b`.

11.4 Danh sách endpoint (tóm tắt)

v1: events (list, SSE stream, ack, clip), sessions (attendance, checks), summary (safety, training), zones (CRUD theo camera), cameras (CRUD, start/stop, stream.mjpg, snapshot), schedules (CRUD), auth (login, password, profile), system (time, health). Legacy vẫn được UI dùng: `/api/faces*`, `/api/zones`, `/api/snapshot`, `/api/schedules*` (POST không validate), `/api/attendance*`, `/api/start|stop|upload*`, `/api/time`, `/api/status`, `/api/alerts`, `WS /ws` (không còn client nhưng server vẫn broadcast frame base64).